

Università di Bologna
C.d.S. Informatica per il Management
Ingegneria del Software - A.A. 2024/25

MANUALE DELLO SVILUPPATORE

MEMBRI:

- ANTONELLA BELEN, CANDIA, Matr: 0001097853
- AURORA, CORTECCHIA, Matr: 0001069738
 - ALESSIA, DIETTI, Matr: 0001097971
 - LORENZO, STABILE, Matr: 0001069691

INDICE

1. Introduzione	3
2. Requisiti per l'installazione	5
3. Setup del progetto	7
4. Struttura del frontend	10
5. Struttura del backend	12
6. Strategie e decisioni implementative	18
7. API REST	23
8. Testing	27
9. Risoluzione problemi e strumenti di debug	29
10. Riferimenti e allegati	33

1. Introduzione

Questo Manuale dello Sviluppatore raccoglie le informazioni tecniche essenziali per comprendere, installare, eseguire e mantenere il progetto *Nota Bene*. È pensato principalmente per sviluppatori esterni (o nuovi membri del team) che devono orientarsi nel codice e lavorare in autonomia, senza dipendere dal contesto storico del gruppo.

Nel manuale vengono descritti:

- la configurazione dell'ambiente di sviluppo;
- l'avvio di backend e frontend;
- la struttura dell'architettura e dei package;
- le scelte progettuali più rilevanti (pattern e motivazioni);
- il funzionamento tecnico delle funzionalità principali;

Si assume una conoscenza di base di Java, Spring Boot, MapDB, HTML/CSS/JavaScript e dei concetti fondamentali di programmazione orientata agli oggetti.

1.1 Scopo del manuale

L'obiettivo di questo documento è fornire una guida operativa, chiara e completa, per permettere a chi legge di:

- comprendere rapidamente come è organizzato il progetto;
- installare e avviare l'applicazione su un nuovo computer;
- individuare dove si trova la logica applicativa (Controller → Service → Repository);
- lavorare sui componenti principali (DTO, modelli, persistenza MapDB, frontend);
- estendere o modificare il sistema mantenendo coerenza con lo stile e le regole definite dal team.

Il manuale serve quindi come riferimento tecnico stabile per garantire manutenibilità, continuità e replicabilità del progetto nel tempo.

1.2 Architettura generale del progetto

L'applicazione *Nota Bene* segue un'architettura client-server composta da due macro-componenti: backend e frontend.

Backend (Maven, Java, Spring Boot, MapDB)

Il backend rappresenta il nucleo logico del sistema ed è organizzato in tre livelli principali:

Controller Layer

- Espone le API REST e gestisce la comunicazione con il frontend.
- I controller validano i parametri e delegano la logica applicativa ai service.

Service Layer

- Contiene la logica di business: gestione note, permessi, condivisioni, versionamento, lock e cartelle.
- Qui vengono applicate le regole che garantiscono coerenza e corretto comportamento del sistema.

Repository Layer

- Gestisce la persistenza tramite MapDB.
- Incapsula tutte le operazioni di lettura/scrittura su file e mantiene il service indipendente dai dettagli del database (Repository Pattern).

La comunicazione tra frontend e backend avviene tramite DTO (Data Transfer Object), che definiscono in modo esplicito il formato di richieste e risposte JSON.

Frontend (Web – HTML/CSS/JavaScript)

Il frontend è un'applicazione web leggera basata su:

- HTML per la struttura delle pagine (login, dashboard, cartelle);
- CSS per lo stile e il layout responsivo;
- JavaScript vanilla per la logica (chiamate API con fetch(), rendering dinamico, gestione modali, lock system lato client).

Non vengono usati framework frontend: l'interfaccia è volutamente semplice, per mantenere il progetto installabile e testabile rapidamente.

Persistenza – MapDB

La persistenza è gestita tramite MapDB, un database embedded basato su file locali (notes.db e users.db).

È stato scelto perché:

- evita la configurazione di un DB server esterno;
- permette persistenza su file con strutture dati tipo mappa (chiave → valore);
- è adatto a un contesto didattico dove si privilegia semplicità di setup e portabilità.

Flusso generale

In termini operativi:

- il frontend invia richieste HTTP (GET/POST/PUT/DELETE) al backend;
- i controller ricevono la richiesta e la trasformano in DTO;
- i service applicano le regole (permessi, lock, versioni...);
- i repository salvano o recuperano i dati da MapDB;
- il backend risponde in JSON e il frontend aggiorna la UI.

Questa struttura è stata mantenuta per garantire separazione delle responsabilità, facilità di manutenzione ed estendibilità.

1.3 Tecnologie utilizzate

L'intero progetto si basa su tecnologie consolidate, sicure e facili da installare.

Backend

- Java 17 (LTS);
- Spring Boot 3.3.x;
- Maven;
- MapDB 3.x;
- JUnit 5;
- Mockito;
- JSON.

Frontend

- HTML5;
- CSS3;
- JavaScript vanilla;
- Risorse.

Strumenti di sviluppo

- Git + GitHub;
- Visual Studio Code;
- Browser Chrome / Edge.

2. Requisiti per l'installazione

Questa sezione raccoglie i prerequisiti necessari per installare ed eseguire *Nota Bene* su un nuovo computer. L'obiettivo è permettere a uno sviluppatore esterno di ottenere rapidamente un ambiente funzionante, pronto sia per l'esecuzione sia per lo sviluppo.

2.1 Software necessario

Per lavorare correttamente sul progetto sono richiesti i seguenti strumenti.

Backend

Java 17 (LTS)

Il backend è sviluppato in Java 17. È possibile usare OpenJDK 17 oppure Oracle JDK 17.

Verifica installazione: `java -version`

Maven 3.8+

Serve per scaricare le dipendenze, compilare ed eseguire l'app Spring Boot.

Verifica installazione: `mvn -version`

IDE consigliato

Visual Studio Code con:

Extension Pack for Java

Spring Boot Extension Pack

Maven for Java

In alternativa, qualunque IDE compatibile con Maven/Spring.

Frontend

Browser moderno (Chrome / Edge consigliati)

Il frontend non richiede build tools né server dedicati per funzionare (può essere avviato anche aprendo i file HTML).

Strumenti aggiuntivi (consigliati)

Git per clonare la repository e lavorare tramite branch/Pull Request.

Chrome / Edge per testare UI e console di debug.

2.2 Clonazione repository GitHub

Per ottenere il codice sorgente eseguire:

1. Aprire il terminale nella directory desiderata;
2. Clonare la repository: `git clone https://github.com/<nome-team>/NotaBene.git`;
3. Entrare nella cartella del progetto: `cd NotaBene`;
4. (Opzionale) Passare al branch principale: `git checkout main`.

Il progetto sarà ora disponibile sul computer per l'esecuzione e la modifica.

2.3 Primo avvio del progetto (quik start)

Dopo aver clonato la repository, il primo passo è verificare che il backend compili correttamente e che i test passino.

Nota: i comandi vanno eseguiti nella cartella backend/ (dove si trova il pom.xml).

1. Entrare nel backend: - `cd backend`
2. Build completa + esecuzione test: - `mvn clean install`
Questo comando:
 - scarica le dipendenze,
 - compila il progetto,
 - esegue i test automatici.
3. Eseguire solo i test (opzione rapida): - `mvn clean test`
4. Avvio dell'app Spring Boot: - `mvn spring-boot:run`

A questo punto, se tutto è settato correttamente, appare sul terminale:

Backend Spring Boot avviato su `http://localhost:8080`

Per arrestare l'applicazione: **CTRL + C** nel terminale.

Per i dettagli completi dell'avvio e della configurazione (VS Code, Maven, MapDB), vedere Sezione 3.

2.4 Struttura del progetto

Il progetto è organizzato in modo da separare chiaramente backend e frontend, con una persistenza locale basata su MapDB.

Backend (Spring Boot)

Percorso principale sorgenti: `backend/src/main/java/com/sweng/notes/`

Componenti principali:

- `controller/` → endpoint REST
- `service/` → logica applicativa
- `repository/` → persistenza MapDB

- model/ → classi dominio
- dto/ → oggetti di scambio per API

Persistenza MapDB

La persistenza utilizza file locali generati nella cartella /data dentro al backend:

- data/notes.db;
- data/users.db.

Questi file contengono lo stato persistente del sistema tra un avvio e l'altro.

Frontend (HTML/CSS/JavaScript)

Percorso: /frontend

Il frontend comunica con il backend tramite fetch() verso le API, ad esempio:

- GET /api/notes/...
- POST /api/notes/...
- PUT /api/notes/...
- DELETE /api/notes/...

Test (JUnit)

I test si trovano in backend/src/test/java/com/sweng/notes/ e includono:

- test sui controller;
- test sulla repository;
- test sui service.

3. Setup del Progetto

Questa sezione descrive come preparare, configurare e avviare il backend di *Nota Bene* su un nuovo computer, e come verificare l'integrazione con il frontend.

Al termine dei passaggi, uno sviluppatore sarà in grado di:

- aprire il progetto in Visual Studio Code;
- installare le dipendenze Maven;
- compilare ed eseguire l'applicazione Spring Boot;
- verificare che le API REST siano raggiungibili;
- comprendere la persistenza MapDB (file .db);
- avviare il frontend in locale e verificare il collegamento Frontend ↔ Backend.

3.1 Ottenere il progetto (Git clone oppure ZIP)

Sono supportate due modalità.

Caso A — Clonazione con Git (consigliata)

Prerequisito: Git installato.

- Aprire Visual Studio Code.
- Aprire la palette comandi: Ctrl + Shift + P.
- Selezionare: Git: Clone.
- Incollare l'URL della repository GitHub (<https://github.com/A-nto26/IngSoft-NotaBene.git>)
- Selezionare una cartella di destinazione.

- Quando VS Code chiede conferma, cliccare Open per aprire la cartella clonata.

Vantaggi:

- aggiornamenti semplici (git pull);
- gestione branch/PR integrata;
- workflow coerente con lo sviluppo del team.

Caso B — Download ZIP (alternativa)

Scaricare il progetto come ZIP da GitHub.

- Estrarre la cartella in una directory locale (es.: C:\Progetti\IngSoft-NotaBene).
- Aprire Visual Studio Code → File → Open Folder → selezionare la cartella estratta.

Nota: in questo caso non si hanno branch e pull automatici (a meno di inizializzare Git manualmente).

3.2 Aprire il progetto in Visual Studio Code (setup consigliato)

Estensioni richieste:

In VS Code installare:

- Extension Pack for Java;
- Spring Boot Extension Pack;
- Maven for Java.

Apertura cartella

Aprire la cartella principale del progetto con:

- File → Open Folder → selezionare la root del progetto (dove sono presenti backend/ e frontend/).

Dopo l'apertura:

- VS Code dovrebbe rilevare automaticamente Maven e proporre l'import del progetto;
- nel pannello laterale dovresti vedere Maven e/o Spring Boot Dashboard.

3.3 Installare dipendenze e compilare (Maven)

Il backend è un progetto Maven. Le dipendenze sono definite nel file pom.xml dentro backend/.

- Aprire il terminale nella cartella backend/ ed eseguire: *mvn clean install*.
 - Questo comando:
 - scarica le dipendenze;
 - compila il progetto;
 - esegue i test.
- Se vuoi eseguire solo i test: *mvn clean test*.

Alternative da VS Code:

- aprire la vista Maven → eseguire i goal clean, install, test;
- oppure ricaricare il progetto Maven dalla Command Palette.

Dipendenze principali usate (dal pom.xml)

- spring-boot-starter-web — API REST / controller
- org.mapdb:mapdb:3.0.9 — persistenza embedded su file
- spring-security-crypto — hashing password (BCrypt)
- com.google.code.gson:gson:2.10.1 — gestione JSON (utility)
- spring-boot-starter-test (scope test) — test, con esclusione di junit-vintage-engine
- Versioni e toolchain
- Spring Boot 3.3.2 (parent)
- Java 17 (source/target 17)
- maven-surefire-plugin 3.1.2 (test)
- spring-boot-maven-plugin (avvio con spring-boot:run)

3.4 Persistenza: cosa sapere su MapDB (notes.db / users.db)

In questo progetto MapDB è un database embedded: non serve installare MySQL/PostgreSQL né configurare server esterni.

I dati vengono salvati in file locali .db e restano disponibili tra un avvio e l'altro.

Creazione automatica dei file

All'avvio, se i file non esistono, l'app:

- crea automaticamente la directory backend/data/ ;
- genera i file di persistenza, nel nostro caso:
 - backend/data/notes.db;
 - backend/data/users.db.

Reset ambiente (ripartire da zero)

Per avere un database vuoto:

1. eliminare i file .db dentro backend/data/;
2. riavviare l'app Spring Boot.

Al successivo avvio, MapDB verrà ricreato vuoto.

3.5 Avvio dell'app Spring Boot

Dalla cartella backend/: *mvn spring-boot:run*.

Se l'avvio va a buon fine: *Backend Spring Boot avviato su http://localhost:8080*

3.6 Avvio del frontend e verifica collegamento Frontend ↔ Backend (manuale)

Il progetto non include test automatici sul frontend: la verifica è manuale.

Avvio del frontend (locale)

Puoi avviarlo in due modi:

Metodo 1 — Avvio diretto dal browser (più semplice)

- Aprire la cartella frontend/.
- Fare doppio clic su: auth.html.

Vantaggi:

- nessuna configurazione.
- immediato e leggero.

Metodo 2 — Live Server

Per ridurre problemi di cache/ricarica:

- In VS Code installare l'estensione Live Server.
- Aprire la cartella frontend/.
- Cliccare Go Live in basso a destra:
- Esempio URL: `http://127.0.0.1:5500/auth.html`

Verifica integrazione:

- Avviare il backend su `http://localhost:8080`.
- Aprire il frontend.
- Provare un'operazione che richiede API:
 - registrazione;
 - login;
 - caricamento note (dashboard).

Se il backend non è avviato o l'URL è errato, il frontend mostrerà un errore tipico (es. "Impossibile connettersi al server").

Questo è un indicatore immediato di:

- backend spento;
- URL/porta errati;
- endpoint non coerenti tra frontend e backend.

4. Struttura del Frontend

Il frontend di Nota Bene è una web application leggera realizzata con HTML, CSS e JavaScript vanilla, pensata per consumare le API REST esposte dal backend Spring Boot.

Non utilizza framework frontend né strumenti di build obbligatori: l'interfaccia è composta da pagine HTML statiche, arricchite da logica JavaScript che genera e aggiorna dinamicamente i contenuti (note, cartelle, badge, versioni, lock, menu, toast).

Il corretto funzionamento delle funzionalità "dinamiche" (login, caricamento note, condivisione, lock, versioni, cartelle) dipende dalla disponibilità del backend su `http://localhost:8080`: se il backend non è raggiungibile o l'URL non è configurato correttamente, il frontend mostrerà errori lato interfaccia, tipicamente "Impossibile connettersi al server".

4.1 Struttura della cartella frontend/

Struttura :

- `css/` : fogli di stile per dashboard, cartelle e layout generale;
- `js/` : logica dell'interfaccia (rendering, fetch, lock/versioni, gestione UI);
- `img/` : asset grafici (icone SVG, immagini UI, risorse statiche).

4.2 File HTML, CSS e JS principali

HTML principali

- `auth.html` : pagina unica per login + registrazione (switch tra sezioni);
- `dashboard.html` : vista principale (note, condivisioni, ricerca, strumenti);
- `cartelle.html` : gestione cartelle (selezione, colori, contenuto).

Le pagine HTML contengono la struttura statica. Il contenuto dinamico (liste note, badge, versioni, cartelle, menu, indicatori lock, ecc.) viene generato e aggiornato via JavaScript.

CSS principali

- auth.css : stile pagina login/registrazione;
- dashboard.css : stile dashboard (card note, tool panel, modali, toast);
- cartelle.css : stile pagina cartelle (sidebar, lista cartelle, note, preview).

L'uso di variabili CSS (:root) per palette colori, ombre e radius garantisce coerenza grafica e facilita la manutenzione.

JS principali

- auth.js: login/registrazione via API, gestione utente loggato (es. loggedInUser in localStorage);
- dashboard.js: cuore dell'app lato UI (caricamento note, CRUD, ricerca, duplicazione, versioni, condivisione, modali, rendering badge/permessi, menu, toast);
- lock-system.js: gestione lock lato interfaccia (timer/popup) e sincronizzazione con backend;
- cartelle.js : gestione cartelle (render, colori, selezione cartella, preview note, menu e azioni come removeSelf/delete).

4.3 Comunicazione con il backend

Il frontend comunica con il backend Spring Boot tramite chiamate REST con fetch(...).

Tutte le chiamate puntano all'host del backend (tipicamente <http://localhost:8080>) e sono organizzate per funzionalità: utenti, note, lock e cartelle.

Endpoint principali usati dal frontend:

Utenti

- POST /api/users/register
- POST /api/users/login
- GET /api/users

Note

- GET /api/notes/visible/{username}
- GET /api/notes/{id}?user=...
- POST /api/notes
- PUT /api/notes/{id}?user=...
- DELETE /api/notes/{id}?user=...
- POST /api/notes/{id}/duplicate?user=...
- PUT /api/notes/{id}/restore/{index}?user=...
- POST /api/notes/{id}/share?user=...
- POST /api/notes/{id}/removeSelf

Lock

- POST /api/notes/{id}/lock?user=...
- POST /api/notes/{id}/lock/refresh?user=...
- POST /api/notes/{id}/unlock?user=...
- GET /api/notes/{id}/lock

Cartelle

- GET /api/folders
- POST /api/folders
- DELETE /api/folders/{nome}
- GET /api/folders/{nome}/user/{username}

Avvertenza pratica su dashboard.js (manutenzione e “conflitti”)

dashboard.js è il file più grande e complesso del frontend: contiene molta logica UI (rendering, modali, menu, lock, versioni, fetch, stato locale).

Per questo motivo può essere anche il punto in cui, durante modifiche parallele o refactoring, è più facile introdurre conflitti o stati temporaneamente incoerenti (es. refresh di componenti, aggiornamento liste, sincronizzazione dopo lock/versioni).

Nota pratica: se durante l'uso noti comportamenti strani o elementi che non si aggiornano (es. lista note, badge, modali), prima di assumere un bug “grave” è consigliato:

- refreshare la pagina e riprovare l'azione;
- verificare che il backend sia raggiungibile (altrimenti compariranno errori di fetch);
- svuotare la cache del browser e fare un “hard refresh” (es. *Ctrl+F5* su Windows / *Cmd+Shift+R* su Mac), oppure pulire cache e dati del sito da Impostazioni del browser se il problema persiste.

5. Struttura del Backend

L'architettura interna del progetto *Nota Bene* è progettata per essere modulare, estensibile e facilmente manutenibile.

Il backend adotta un modello a livelli (Controller → Service → Repository) mentre il frontend comunica tramite API REST usando JavaScript vanilla.

In questa sezione vengono descritti:

- organizzazione dei package backend;
- responsabilità di ogni file principale;
- persistenza con MapDB;
- gestione permessi (Strategy);
- versionamento;
- lock system.

5.1 Panoramica package backend

controller/

Breve descrizione

Contiene i REST Controller che espongono gli endpoint pubblici. Si occupano di:

- validare parametri minimi (es. user mancante, campi obbligatori);
- ricevere DTO dal frontend;
- chiamare i Service;
- ritornare risposte HTTP coerenti (200/400/403/404/409).

File

FolderController.java

Controller REST per le cartelle (/api/folders). Espone:

- GET /api/folders → restituisce tutte le cartelle come FolderResponse (mapping da Cartella);
- POST /api/folders → crea una cartella (controlla nome obbligatorio, gestisce conflitto 409 se già esistente);
- DELETE /api/folders/{nome} → elimina una cartella e dissocia le note;
- GET /api/folders/{nome}/user/{username} → ritorna solo le note visibili per quell'utente in quella cartella.
- Usa CORS permissivo e delega tutte le regole applicative a FolderService.

NoteController.java

- Controller REST per le note (/api/notes). Espone:
- GET /api/notes/{id}?user=... → restituisce una NoteView (nota filtrata e "arricchita" per il frontend);
- GET /api/notes/visible/{username} → restituisce le note visibili (mie + condivise) come lista di NoteView;
- POST /api/notes → crea una nota (titolo/contenuto/creatore obbligatori)
- PUT /api/notes/{id}?user=... → modifica nota con:
 - controllo permessi (puoScrivere);
 - controllo lock (409 se scaduto o detenuto da altro);
 - controllo versione (versionExpected) per prevenire overwrite (409);
 - update delegato a NoteService;
 - gestione aggiunta utenti solo se autore e solo come aggiunta;
 - rilascio lock a fine update;
- DELETE /api/notes/{id}?user=... → elimina nota (solo autore);
- PUT /api/notes/{id}/restore/{index}?user=... → ripristina versione (autore o scrittura, blocco se lock di altri);
- POST /api/notes/{id}/share?user=... → condivide la nota (solo aggiunta utenti);
- POST /api/notes/{id}/removeSelf → utente condiviso si rimuove da solo dalla nota;
- POST /api/notes/{id}/duplicate?user=... → duplica nota (se può leggere);
- Locking:
- POST /api/notes/{id}/lock?user=...

- POST /api/notes/{id}/lock/refresh?user=...
- POST /api/notes/{id}/unlock?user=...
- GET /api/notes/{id}/lock

Include utility interne di normalizzazione username (trim + lowercase) e check autore (isAutore).

UserController.java

Gestisce API utenti:

- POST /api/users/register registrazione con validazione campi
- POST /api/users/login login con gestione errori (404 utente, 401 password, 400 input)
- GET /api/users elenco username (utile per UI condivisione)

Include normalizzazione username (trim + lowercase) e configurazione CORS permissiva.

service/

Breve descrizione

Contiene la logica applicativa vera e propria. Qui vengono applicate le regole del dominio:

- permessi e condivisioni;
- versionamento e restore;
- validazione input “di business”;
- lock concorrente (acquisizione, refresh, unlock);
- gestione cartelle e filtro visibilità note per utente.

File

NoteService.java

Servizio principale delle note. Implementa:

- Creazione (create) con permesso iniziale e inizializzazione completa (versioni vuote, lock nullo, timestamp, colore cartella);
- Visibilità (mie + condivise) con filtri (getVisibleNotesForUser, getNotesForUserFiltered);
- NoteView (toView) per restituire al frontend informazioni “pronte”:
 - permesso (stringa);
 - ruolo (autore/scrittura/lettura/hidden);
 - lockedBy (lock effettivo solo se valido);
 - versione corrente e limite versioni;
 - lista utenti condivisi filtrata (esclude autore e utente corrente);
- Update (update) con:
 - check permessi (puoScrivere);
 - check lock effettivo (scaduto → conflict);
 - salvataggio versione precedente solo se ci sono modifiche reali;
 - aggiornamento metadati lastModifiedAt/By;
 - supporto cambio permesso (se presente in request) con vincolo: non tornare “Privata” se la nota è condivisa;

- Delete solo autore;
- Duplicate: copia privata, senza condivisioni, senza versioni;
- Share: autore può solo aggiungere utenti;
- RemoveSelf: utente condiviso può rimuovere solo sé stesso;
- RestoreVersion: autore o scrittura, blocco se lock di altri;
- Locking: lock, unlock, refreshLockState, getLockOwner, getLockState;

FolderService.java

Gestisce cartelle e note per cartella:

- getAllFolders;
- createFolder con colore default e gestione conflitto;
- deleteFolder;
- getNotesInFolderForUser: filtra le note di una cartella in base a Note.puoLeggere(user);

UserService.java

Gestisce autenticazione base:

- register con validazione username + password minima, hashing BCrypt;
- login con encoder.matches;
- getAllUsers / getAllUsernames per UI di condivisione.

repository/

Breve descrizione

Gestisce la persistenza tramite MapDB e fornisce metodi CRUD e query.

È separato dalla logica applicativa: non decide permessi o regole di business, ma garantisce consistenza, commit e retrocompatibilità dei dati salvati.

File

NoteRepository.java

Persistenza su data/notes.db con mappe:

- notes (id → Note);
- folders (nome → Cartella);
- Funzioni: findAll, findById, save, delete;
- normalizzazione note all'avvio (normalizzaNote) per retrocompatibilità (campi null, versioni, permessi, lock, colore);
- gestione cartelle: creazione, eliminazione, note per cartella;
- sincronizzazione colore cartella/note;
- lock tecnico: tryLock, refreshLock, unlockNote, getEffectiveLockOwner, getLockState;
- condivisione: addUsersToShare, removeSelf, findSharedWithUser.

UserRepository.java

Persistenza su data/users.db con mappa:

- users (username → Utente);
- save con validazioni e commit;
- Funzioni: findByUsername, exists, findAll;
- normalizzazione username case-insensitive.

model/

Breve descrizione

Classi di dominio che rappresentano lo stato applicativo.

Includono logica coerente col dominio (es. `puoLeggere`, snapshot versioni, helper lock), evitando logica da controller/service.

File

`Note.java`:

- contenuto, condivisione, permesso, versioni, lock, metadati, colore cartella;
- include `salvaVersionePrecedente()` (max 50) e metodi `puoLeggere/puoScrivere`.

`VersioneNota.java`:

- snapshot di titolo/contenuto + timestamp.

`Cartella.java`:

- nome/creatore/colore/createdAt + notelds per associare note.

`Utente.java`:

- username normalizzato + passwordHash BCrypt.

`Permesso.java`:

- strategia permessi; implementazioni:
`Privata.java`;
`Lettura.java`;
`Scrittura.java`.

dto/

Breve descrizione

DTO per richieste/risposte API: separano dominio e API pubblica e stabilizzano il contratto JSON.

File

`CreateNoteRequest.java`:

- dati per creare una nota (permesso scelto in creazione, utenti iniziali, colore cartella).

`NoteUpdateRequest.java`:

- update nota (titolo/contenuto/cartella/colore, `versionExpected`, permesso opzionale).

`NoteView.java`:

- risposta "frontend-friendly" (ruolo, permesso, `lockedBy`, versione, limit, utenti filtrati, metadati, versioni).

`ShareNoteRequest.java`:

- aggiunta utenti alla condivisione (normalizzazione lowercase).

`FolderRequest.java`:

- create folder (nome, creatore, colore).

FolderResponse.java:

- rappresentazione cartella per output API (derivata da Cartella).

UserRequest.java / UserResponse.java:

- richieste e risposte autenticazione.

PermessoRequest.java:

- DTO dedicato al cambio permesso (se usato da endpoint specifici).

5.2 Controller → Service → Repository

Il backend segue un'architettura a tre livelli:

Livello Controller

Riceve richieste HTTP e gestisce:

- Validazioni minime (campi obbligatori, user mancante);
- Conversione JSON → DTO;
- Mapping DTO → Service;
- Codici HTTP coerenti (es. 403 permessi, 409 lock/versione, 404 not found).

Esempi reali:

PUT /api/notes/{id}?user=... controlla permesso + lock + versionExpected prima di chiamare il service.

POST /api/folders gestisce conflitto 409 se cartella esistente.

Livello Service

Contiene la logica applicativa centrale:

- regole permessi e ruoli (autore / condiviso lettura/scrittura);
- versionamento automatico e restore;
- lock concorrente con timeout e refresh;
- gestione cartelle e filtro visibilità per utente;
- conversione Note → NoteView per il frontend.

Livello Repository

Gestisce MapDB e la consistenza dei dati:

- commit su file;
- normalizzazione dati al boot (retrocompatibilità);
- query ottimizzate e CRUD;
- gestione lock tecnico e cartelle persistenti.

5.3 Persistenza con MapDB (note, cartelle, utenti)

MapDB è usato come database embedded per memorizzare lo stato applicativo su file locali, senza dipendenze da un DBMS esterno.

Strutture dati effettive nel progetto

data/notes.db:

- mappa notes → id → Note
- mappa folders → nomeCartella → Cartella

data/users.db:

- mappa users → username → Utente

Caratteristiche principali

- persistenza su file locali;
- accesso key-value semplice e veloce;
- commit esplicito per garantire consistenza dei dati.

Supporto a debugging e tracciamento

Per facilitare debugging e verifica della persistenza durante lo sviluppo, sono stati introdotti:

- un sistema di logging (per tracciare eventi e flussi rilevanti, es. lock/versioning/errori);
- strumenti interni di ispezione del DB (DBInspector) per leggere e verificare rapidamente il contenuto delle mappe persistenti.

I dettagli su log e DBInspector (uso pratico, esempi e output) vengono descritti nella sezione dedicata a Testing e Debugging.

5.4 DTO per comunicazione API

I DTO sono fondamentali per:

- non esporre direttamente i modelli interni;
- standardizzare i payload;
- semplificare controller e frontend.

Esempi:

- CreateNoteRequest per creazione nota;
- NoteUpdateRequest per update con versionExpected;
- NoteView per output completo e filtrato verso frontend;
- FolderRequest/FolderResponse per cartelle;
- UserRequest/UserResponse per autenticazione.

6. Strategie e decisioni implementative

Questa sezione raccoglie le principali scelte progettuali e strategie di implementazione adottate in *Nota Bene* per gestire in modo coerente e sicuro i processi core dell'applicazione: permessi e condivisioni, versionamento e restore, locking concorrente, e le azioni principali (creazione/modifica/eliminazione/duplicazione/cartelle/ricerca).

L'obiettivo è descrivere come e perché il sistema si comporta così, evitando ripetizioni con la sezione 5 (architettura backend e struttura dei package).

6.1 Separazione delle responsabilità: Controller → Service → Repository

Decisione: mantenere i Controller "sottili" e spostare la logica di business nei Service, con persistenza incapsulata nei Repository.

- Controller: validazioni minime (parametri obbligatori), conversione JSON→DTO, gestione codici HTTP coerenti (400/401/403/404/409).
- Service: regole applicative (permessi, restore, lock, versionamento, condivisioni, filtri visibilità).

- Repository: sola persistenza MapDB + consistenza/commit + normalizzazione retrocompatibilità.

Vantaggi:

- codice più testabile (service mockando repository),
- manutenzione più semplice,
- riduzione di logica duplicata nei controller.

6.2 Permessi e condivisioni: Strategy Pattern + regole di dominio

Il progetto modella i permessi tramite Strategy Pattern (polimorfismo), evitando catene lunghe di if/else.

Modello:

- Permesso (astratto) + implementazioni: Privata, Lettura, Scrittura
- ogni permesso definisce comportamento standard:
 - puoLeggere(),
 - puoScrivere().

Regole applicative

Autore:

- può aggiungere utenti alla condivisione (non può rimuoverli).
- può modificare titolo/contenuto/cartella.
- può modificare il permesso anche dopo la creazione, con vincoli:
 - può passare:
 - privata → lettura/scrittura,
 - e lettura ↔ scrittura;
- non può tornare a privata se esistono utenti condivisi (evita invisibilità “forzata” agli utenti condivisi senza rimozione formale).

Utenti condivisi:

- possono leggere se il permesso lo consente (lettura/scrittura),
- possono scrivere solo se permesso = scrittura,
- possono rimuovere solo sé stessi dalla condivisione (removeSelf), perché questa azione non impatta gli altri utenti.

DTO dedicato al permesso

PermessoRequest (quando usato) serve a inviare un cambio permesso “pulito” e isolato (solo campo permesso, normalizzato), evitando di mischiare intenzioni diverse nella stessa request.

6.3 Versionamento e restore: snapshot coerenti e regole di sicurezza

Ogni modifica “rilevante” (contenuto/titolo/cartella) può salvare automaticamente una versione precedente come snapshot:

- VersioneNota(titolo, contenuto, timestamp)
- inserita in testa alla lista (più recente per prima)
- limite massimo: 50 versioni
- la UI mostra la versione corrente come: size(versioni) + 1

Regole di restore

Restore consentito a:

- Autore oppure
- Utente condiviso con permesso scrittura
- Restore NON consentito agli utenti con permesso lettura:
l'azione viene bloccata e l'utente riceve un messaggio di stop/errore ("permessi insufficienti").
- Restore bloccato durante lock detenuto da un altro utente:
in quel caso l'utente può solo visualizzare l'anteprima.
- Il restore non modifica il permesso corrente della nota: ripristina titolo/contenuto (e metadati previsti), ma il permesso resta quello attuale.
- il versionamento è consentito per un massimo di 50 volte, al raggiungimento di tale limite l'utente riceverà una notifica, le modifiche non verranno salvate.

Aggiornamento contenuto per utenti condivisi

Gli utenti in condivisione (lettura/scrittura), quando:

- fanno refresh manuale oppure
- aprono l'anteprima della nota

se la nota è stata aggiornata nel frattempo ricevono un toast/info di aggiornamento e vedono i dati aggiornati.

Decisione UX importante: informare l'utente della sincronizzazione, evitando "mismatch silenziosi".

6.4 Lock system: concorrenza controllata e azioni bloccate durante lock

Il lock impedisce modifiche concorrenti garantendo che una sola sessione di editing sia attiva per volta.

Funzionamento backend

Quando l'utente apre la modale di modifica:

POST /api/notes/{id}/lock?user=...

Comportamento:

- se la nota è libera (o il lock precedente è scaduto) → lock assegnato
- se la nota è già lockata da un altro utente → 409 Conflict con indicazione lockedBy

Il lock è legato a:

- username proprietario del lock
- lockedAt timestamp

Timeout backend (safety timeout)

Il lock scade automaticamente dopo 10 minuti dall'ultima attività valida (lockedAt), per evitare blocchi permanenti in caso di crash, chiusura della pagina o perdita di connessione. In caso di scadenza, il lock viene considerato non valido e può essere riassegnato.

Timeout frontend (scelta UX)

La UI applica un flusso guidato per ridurre i lock “dimenticati”, senza dipendere dal timeout backend:

- timer iniziale 3 minuti
- allo scadere compare un popup di conferma (2 minuti) “Stai ancora modificando la nota?”
 - Sì → refresh del lock e prosecuzione
 - No / nessuna risposta → chiusura della modale, auto-salvataggio solo se ci sono modifiche reali, e rilascio lock

Endpoint di supporto:

- POST /api/notes/{id}/lock/refresh?user=... (rinnovo lock se posseduto)
- POST /api/notes/{id}/unlock?user=... (rilascio lock)
- GET /api/notes/{id}/lock (stato lock)

Effetti del lock sulle azioni utente

Durante un lock detenuto da un altro utente, tutte le azioni che modificano lo stato della nota vengono bloccate. In particolare:

- restore versioni disabilitato → l’utente può solo visualizzare l’anteprima
- eliminazione bloccata → l’autore non può eliminare la nota se lockata da un altro utente (es. utente in scrittura)
- removeSelf consentito anche durante lock → l’utente condiviso può rimuovere solo sé stesso perché non altera lo stato per gli altri utenti (operazione “locale” rispetto alla sua visibilità)

6.5 Gestione delle cartelle

La pagina Cartelle consente agli utenti di visualizzare, navigare e interagire con le cartelle e le note al loro interno.

Navigazione e anteprima

- L’utente può navigare tra le cartelle e visualizzare in anteprima le note contenute.
- In anteprima viene mantenuta la coerenza tra versione visualizzata e versione corrente della nota: all’apertura dell’anteprima (o al refresh manuale) viene verificata la versione lato server e, se la nota è stata aggiornata, la UI mostra un messaggio informativo e aggiorna il contenuto mostrato.

Azioni disponibili

- Eliminazione nota: consentita solo all’autore (e comunque soggetta ai vincoli di lock).
- Rimozione dalla condivisione (removeSelf): consentita agli utenti condivisi anche dalla pagina cartelle; non influisce sugli altri utenti e non elimina la nota per l’autore.

Creazione nuova cartella (flusso UX)

- L’utente può aggiungere una nuova cartella: si apre un pop-up che richiede il nome della cartella.

- Confermato il nome, l'utente viene reindirizzato alla dashboard e si apre automaticamente la modale "Nuova nota", con il campo Cartella già precompilato con il nome inserito nella pagina Cartelle.
- Questo riduce i passaggi e rende immediata la creazione di una nota "dentro" la nuova cartella.

Vantaggi:

- esperienza utente più fluida
- meno passaggi per aggiungere note alle cartelle
- preview coerente con la versione corrente della nota

6.6 Azioni principali: scelte implementative (overview)

Qui sotto le scelte chiave sulle azioni più importanti:

Creazione nota (POST /api/notes)

- inizializzazione completa (metadati, permesso, versioni vuote, lock nullo)
- persistenza MapDB con ID generato lato repository

Modifica nota (PUT /api/notes/{id}?user=...)

- check permessi (autore o utente condiviso con permesso scrittura)
- check lock (409 se lock di altri)
- check conflitti/versione (se previsto dal flusso con versionExpected)
- versionamento automatico solo se ci sono modifiche reali
- update metadati lastModifiedAt/By

Eliminazione (DELETE /api/notes/{id}?user=...)

- solo autore
- bloccata se lock detenuto da altro utente

Duplicazione (POST /api/notes/{id}/duplicate?user=...)

- permessa a chi può almeno leggere
- nuova nota privatadi default, il permesso è modificabile a piacere, senza condivisioni, senza versioni, versione iniziale 1
- titolo con suffisso "(copia)" (es. Spesa (Copia))

Cartelle

- cartella gestita coerentemente con assegnazione/rimozione note
- eliminazione cartella quando vuota (secondo logica repository/service)

Ricerca e visibilità

- filtro su note visibili (mie + condivise)
- ricerca case-insensitive su titolo/contenuto/cartella
- output "frontend-friendly" tramite NoteView

7. API REST

L'applicazione Nota Bene espone API REST utilizzate dal frontend per gestire note, utenti e cartelle.

Le API sono stateless, scambiano dati in JSON (quando presente un body) e usano codici HTTP standard per comunicare l'esito delle operazioni.

Questa sezione descrive:

- struttura generale delle API;
- endpoint principali;
- esempi di request/response;
- codici errore e gestione dei casi anomali.

7.1 Struttura generale

Le API sono organizzate in tre macro-aree:

- `/api/notes` → note: creazione, lettura, modifica, duplicazione, restore versioni, condivisione, `removeSelf`, lock.
- `/api/users` → utenti: registrazione, login, elenco username.
- `/api/folders` → cartelle: elenco, creazione, eliminazione, note per cartella e utente.

Formato richieste

- Tutte le richieste con body usano `Content-Type: application/json`.
- L'“identità” dell'utente è gestita dal frontend (es. `localStorage`) e viene passata:
 - come query param `?user=...` su molte operazioni note (`update/delete/lock/restore/share/duplicate`),
 - oppure nel body (es. `removeSelf`),
 - oppure come path variable (es. `visible/{username}`, `folders/{nome}/user/{username}`).

7.2 Endpoints Note (`/api/notes`)

Lettura singola nota (`NoteView`)

- `GET /api/notes/{id}?user={username}`

Risposte principali:

- 200 OK → ritorna `NoteView`
- 404 Not Found → nota inesistente

Lettura note visibili (`NoteView`)

- `GET /api/notes/visible/{username}`

Restituisce tutte le note visibili: proprie + condivise, già convertite in `NoteView`.

Risposte principali:

- 200 OK → lista di `NoteView`
- 400 Bad Request → username mancante/non valido

Creazione nota

- POST /api/notes

Body: CreateNoteRequest

Risposte principali:

- 200 OK → ritorna un messaggio testuale (es. "Nota creata...")
- 400 Bad Request → campi obbligatori mancanti (titolo/contenuto/creatore)

Aggiornamento nota (con lock + versionExpected)

- PUT /api/notes/{id}?user={username}

Body: NoteUpdateRequest

Include:

- check permessi scrittura,
- check lock owner (con conflitto se lock non valido o di altri),
- check versionExpected per evitare overwrite.

Risposte principali:

- 200 OK → messaggio "Nota aggiornata..."
- 403 Forbidden → niente permessi
- 404 Not Found → nota inesistente
- 409 Conflict → lock scaduto / lock di altri / conflitto versione

Eliminazione nota

DELETE /api/notes/{id}?user={username}

Regole applicate dal controller:

- solo autore.

Risposte principali:

- 200 OK → messaggio "Nota eliminata..."
- 403 Forbidden → non autore
- 404 Not Found → nota inesistente

Ripristino versione

PUT /api/notes/{id}/restore/{index}?user={username}

Regole:

- autore oppure utente con permesso di scrittura,
- bloccato se lock di un altro utente.

Risposte principali:

- 200 OK → "Versione ripristinata."
- 400 Bad Request → indice non valido
- 403 Forbidden → permessi insufficienti
- 404 Not Found → nota inesistente
- 409 Conflict → lock di altri / conflitti di stato

Condivisione (solo aggiunta utenti)

POST /api/notes/{id}/share?user={autore}

Body: ShareNoteRequest (solo lista utenti; il permesso non viene cambiato)

Risposte principali:

- 200 OK
- 400 Bad Request → lista utenti vuota
- 403 Forbidden → non autore
- 404 Not Found → nota inesistente

RemoveSelf (uscita dalla condivisione)

POST /api/notes/{id}/removeSelf

Body: { "user": "username" }

Risposte principali:

- 200 OK
- 400 Bad Request → user mancante
- 403 Forbidden → la nota non è condivisa con quell'utente
- 404 Not Found → nota inesistente

Duplicazione nota

POST /api/notes/{id}/duplicate?user={username}

Risposta: ritorna direttamente una Note (JSON).

Risposte principali:

- 200 OK → Note duplicata
- 403 Forbidden → permessi insufficienti
- 404 Not Found → nota inesistente

Locking

POST /api/notes/{id}/lock?user={username}

Risposte principali:

- 200 OK → {status: "locked" | "expired_recovered"}
- 409 Conflict → {status: "already_locked", lockedBy: "..."}

POST /api/notes/{id}/lock/refresh?user={username}

Risposte principali:

- 200 OK → {status: "refreshed"}

POST /api/notes/{id}/unlock?user={username}

Risposte principali:

- 200 OK → {status: "unlocked"}
- 403 Forbidden → {status: "forbidden"} (se non owner e non scaduto)

GET /api/notes/{id}/lock

Risposte principali:

- 200 OK → stato lock {locked: true/false, lockedBy?, lockedAt?}

7.3 Endpoints Utenti (/api/users)

Registrazione

POST /api/users/register

Body: UserRequest

Risposte principali:

- 200 OK → UserResponse(success=true, ...)
- 400 Bad Request → campi mancanti
- 409 Conflict → username già registrato

Login

POST /api/users/login

Body: UserRequest

Risposte principali:

- 200 OK
- 400 Bad Request → campi mancanti
- 404 Not Found → utente non registrato
- 401 Unauthorized → password errata (o caso generico)

Lista username (per condivisione)

GET /api/users

Risposta: Collection<String> (username già normalizzati/lowercase)

7.4 Endpoints Cartelle (/api/folders)

Lista cartelle

GET /api/folders

Risposta: List<FolderResponse>

Creazione cartella

POST /api/folders

Body: FolderRequest (nome, colore, creatore)

Risposte principali:

- 200 OK → messaggio testuale
- 400 Bad Request → nome mancante
- 409 Conflict → cartella già esistente (IllegalArgumentException)

Eliminazione cartella

DELETE /api/folders/{nome}

Risposta: messaggio testuale 200 OK

Note in cartella per utente

GET /api/folders/{nome}/user/{username}

Risposta: List<Note> (non NoteView)

7.5 Codici errore e gestione eccezioni

- 200 OK → operazione completata
- 400 Bad Request → input non valido o campi mancanti
- 401 Unauthorized → credenziali errate (login)
- 403 Forbidden → violazione permessi (non autore / non scrittura / operazione non consentita)
- 404 Not Found → risorsa inesistente (nota/utente)
- 409 Conflict → conflitto di concorrenza (lock attivo, lock scaduto, conflitto versione)
- 500 Internal Server Error → eccezioni non gestite

8. Testing

Il progetto *Nota Bene* include una suite di test automatici basati su JUnit 5 e Mockito, con l'obiettivo di verificare in modo semplice e rapido il funzionamento delle componenti principali (Controller, Service, Repository).

I test sono pensati per essere chiari, brevi e comprensibili anche a un livello principiante, senza l'obiettivo di coprire ogni singolo ramo del codice.

8.1 JUnit 5 – configurazione

Il progetto utilizza JUnit 5 (JUnit Jupiter) come framework principale di testing, insieme a Mockito per i mock.

Nel pom.xml sono presenti le dipendenze per:

- JUnit Jupiter (JUnit 5) → struttura dei test e assertion
- Mockito → mock di service/repository nei test
- Spring Test (se presente) → utilità per test (anche se qui usiamo test semplici)

Struttura cartella test

I test si trovano in: `src/test/java/...`

JUnit rileva automaticamente tutte le classi che terminano con `*Test.java`.

8.2 Test unitari per i Service

I Service contengono la logica applicativa (es. filtri, validazioni, regole di permesso, ecc.).

Nel progetto sono presenti test unitari semplici per:

- NoteService

Cosa viene verificato (in modo base):

- creazione nota (campi principali valorizzati, salvataggio su repository)
- operazioni principali come duplicazione / rimozione utente dalla condivisione
- funzioni di supporto (visibilità note, ecc.) quando possibile con repository mockato

- FolderService

Test base su:

- ritorno cartelle (delegato al repository);
- creazione cartella (nome obbligatorio, duplicati);
- lettura note per cartella filtrate per utente (solo note leggibili).

- UserService
Test base su:
 - registrazione (ok / username già esistente / vincoli minimi);
 - login (ok / password errata / utente non trovato).

8.3 Mock del repository

Per evitare dipendenze dal DB reale durante i test, nei test di unità vengono usati i mock di repository e service tramite Mockito.

Vantaggi:

- test più veloci;
- nessun accesso a file .db;
- comportamento controllabile (si simulano facilmente casi “ok”, “not found”, “error”).

Esempio tipico (concettuale):

- NoteRepository repo = mock(NoteRepository.class);
- NoteService service = new NoteService(repo);
- when(repo.findById(1)).thenReturn(fakeNote).

8.4 Test dei Controller

I Controller vengono testati in modo semplice tramite unit test con service mockato (Mockito), senza usare MockMvc.

Cosa si verifica tipicamente:

- status code della ResponseEntity (200 / 400 / 404 / 409 quando applicabile);
- che il controller chiami il metodo giusto del service (verify(...));
- che il body non sia null quando serve.

Esempi di test presenti nel progetto:

- NoteControllerTest (create, visible, update base, delete, share, duplicate, lock);
- FolderControllerTest (getAll, create, delete, getNotesInFolderForUser);
- UserControllerTest (register, login, elenco utenti).

8.5 Come eseguire l'intera suite

Metodo 1 — VS Code (Testing Panel)

1. Aprire il progetto backend in VS Code
2. Sezione Testing
3. Run Tests (o Run All)

Metodo 2 — Maven da terminale (consigliato)

1. Da cartella backend/: *mvn clean test*
2. In alternativa: *mvn test*

Output atteso:

- tutti i test passano (stato GREEN)

report generato in: target/surefire-reports/

Esempio reale (atteso):

Tests run: XX, Failures: 0, Errors: 0, Skipped: 0

Oltre all'esecuzione locale dei test (da VS Code o da terminale con Maven), il progetto integra anche una verifica automatica tramite **GitHub Actions** (CI). In particolare è presente un workflow (.github/workflows/maven.yml) che, ad ogni **push** e ad ogni **Pull Request** verso i branch principali (main/develop), avvia in automatico un ambiente con **Java 17** e lancia la build con test del backend tramite Maven (clean verify). Questo permette di intercettare subito problemi di compilazione o test falliti, riducendo il rischio di integrare modifiche instabili e rendendo la fase di review più affidabile.

Questi controlli risultano particolarmente utili durante la **Pull Request**: appena la PR viene aperta, i check partono automaticamente e GitHub mostra l'esito ( passing /  failing). Finché i controlli non risultano “verdi”, la PR non è considerata pronta e il pulsante di **merge** rimane bloccato/non disponibile secondo le regole del repository; quando invece build e test passano, il merge “si sblocca”, garantendo che l'integrazione su main avvenga solo dopo una validazione automatica. A supporto visivo, nel repository sono stati aggiunti anche i **badge** (es. “Java CI – Maven Build: passing” e “Java 17”), utili per mostrare a colpo d'occhio lo stato della pipeline e la versione di Java prevista.

9. Risoluzione problemi e strumenti di debug

Questa sezione raccoglie gli errori più comuni riscontrabili durante lo sviluppo di Nota Bene e descrive gli strumenti interni introdotti per facilitare il debugging: logging JSONL e DB Inspector MapDB.

9.1 Errori MapDB comuni

MapDB è un database embedded leggero e veloce, ma può generare errori specifici quando i file DB sono bloccati, corrotti o incompatibili con i model Java aggiornati.

Errore: “File locked by another process”

Causa: notes.db o users.db è aperto da un processo precedente (es. run precedente non terminato correttamente).

Soluzioni:

- Chiudere eventuali processi Java/Spring Boot ancora attivi.
Comando: *Ctrl +C*
- Riavviare l'IDE/terminale.
- Se presenti, eliminare eventuali file temporanei MapDB (es. .p / .t associati al DB).
- Riavviare l'applicazione.

Errore: “Serialization error”

Causa: sono cambiati i model Java serializzati in MapDB (aggiunta/rimozione/modifica campi o tipi), e i file DB esistenti contengono oggetti salvati col vecchio schema.

Soluzione (sviluppo locale):

Eliminare i DB locali e farli rigenerare:

- backend/data/notes.db
- backend/data/users.db
- Riavviare Spring Boot (MapDB ricrea i file automaticamente).

Nota: nel progetto è presente una normalizzazione nel repository che riduce alcuni problemi (es. campi null), ma non può risolvere tutte le incompatibilità di serializzazione.

MapDB “non salva” i dati

Causa tipica: assenza di db.commit() dopo operazioni CRUD.

Soluzione:

- Verificare che ogni operazione di modifica chiami commit() nel repository.
- Verificare che db.close() non venga chiamato prematuramente.

9.2 API non raggiungibili

Errore 404 / 405 su chiamate REST

Possibili cause:

- backend non avviato (porta 8080)
- endpoint cambiato nei controller
- URL errato nel frontend

Soluzioni:

- Verificare che Spring Boot sia attivo su http://localhost:8080
- Controllare i mapping nei controller (es. @RequestMapping("/api/notes"))
- Verificare nel frontend: `const API_NOTES = "http://localhost:8080/api/notes";`

Errore: “Failed to fetch”

La richiesta non è arrivata al server.

Cause comuni:

- backend crashato / non avviato
- CORS / origin “null”
- URL digitato male

Soluzioni:

- controllare i log Spring Boot
- aprire la rete DevTools (tab Network) e verificare la request

9.3 CORS / fetch falliti

Se il frontend viene aperto tramite file:///..., il browser usa un'origine null, e possono emergere problemi (specie con alcune policy del browser).

Errore tipico:

- No 'Access-Control-Allow-Origin' header...
- oppure blocchi “origin null”

Soluzioni consigliate:

Avviare un piccolo server locale con l' estensione VS Code Live Server

Nel backend il CORS è già configurato con `@CrossOrigin(...)` sui controller.

9.4 Problemi lock system

Il lock serve per evitare modifiche concorrenti.

Lock non si attiva

Cause tipiche:

- non viene chiamato `POST /api/notes/{id}/lock?user=...`
- parametro user mancante
- nota già lockata

Soluzioni:

- verificare il risultato della chiamata di lock (200 vs 409)
- verificare lo stato lock via: `GET /api/notes/{id}/lock`

Lock non si rilascia

Cause tipiche:

- l'utente che prova a sbloccare non è owner
- il frontend non chiama unlock a fine modifica/chiusura modale
- refresh lock non correttamente gestito lato JS

Soluzioni:

1. verificare che su "Salva" o "Chiudi" venga chiamato:
`POST /api/notes/{id}/unlock?user=...`
2. controllare lo stato con: `GET /api/notes/{id}/lock`

9.5 Versioni non aggiornate

La nota sembra salvata ma la versione non aumenta

Cause tipiche:

- non ci sono modifiche reali (titolo/contenuto/cartella/colore identici)
- la versione non viene salvata perché il limite (max 50) è raggiunto
- PUT fallita (400 / 409 / 403)
- frontend non aggiorna il badge versione

Soluzioni:

- controllare la risposta HTTP della PUT
- controllare il contenuto di `note.versioni` e il campo `versione` in `NoteView`
- assicurarsi che dopo salvataggio il frontend ricarichi la nota/card (es. `caricaNote()`)

9.6 Logging JSONL delle azioni

Il progetto include un logging applicativo semplice tramite `LoggerActions`, che salva eventi in formato JSONL (una riga JSON per evento) dentro la cartella:

logs/

file del tipo: `actions-YYYY-MM-DD.json`

Ogni riga contiene:

- timestamp
- event

- user
- details (mappa con info aggiuntive)

Esempio di visualizzazione:

```
[{"timestamp": "2025-12-13T13:15:43.818312", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:15:46.509796900", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UPDATE_SUCCESS"},
{"timestamp": "2025-12-13T13:15:46.518921500", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:15:46.817029200", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:16:00.161155300", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:16:06.277153500", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UPDATE_SUCCESS"},
{"timestamp": "2025-12-13T13:16:06.285819800", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:16:06.576921100", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:16:11.302721700", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:16:17.018764600", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UPDATE_SUCCESS"},
{"timestamp": "2025-12-13T13:16:17.026784700", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:16:17.314371900", "details": {"noteId": 47, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:25:21.474047300", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_CREATE_SUCCESS"},
{"timestamp": "2025-12-13T13:25:37.736933600", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:25:42.364059500", "details": {"noteId": 48, "usersAdded": [{"testfinale2"}], "user": "provafinale", "event": "NOTE_SHARE_ADD_USERS"},
{"timestamp": "2025-12-13T13:25:44.613087400", "details": {"noteId": 48, "usersAdded": [{"testfinale2"}, {"testfinale02"}], "user": "provafinale", "event": "NOTE_SHARE_ADD_USERS"},
{"timestamp": "2025-12-13T13:25:55.488564600", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:25:58.733739800", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:26:00.347121200", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:32:36.739443500", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:33:18.861385200", "details": {"noteId": 48, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"},
{"timestamp": "2025-12-13T13:33:44.452844300", "details": {"noteId": 49, "user": "provafinale", "event": "NOTE_CREATE_SUCCESS"},
{"timestamp": "2025-12-13T13:33:52.998633500", "details": {"noteId": 49, "user": "provafinale", "event": "NOTE_LOCK_RECOVERED"},
{"timestamp": "2025-12-13T13:34:00.107923", "details": {"noteId": 49, "usersAdded": [{"provafinale3"}], "user": "provafinale", "event": "NOTE_SHARE_ADD_USERS"},
{"timestamp": "2025-12-13T13:34:00.857375800", "details": {"noteId": 50, "user": "provafinale", "event": "NOTE_CREATE_SUCCESS"},
{"timestamp": "2025-12-13T13:34:00.871211100", "details": {"noteId": 49, "user": "provafinale", "event": "NOTE_UNLOCK_SUCCESS"}]
```

Questo sistema è utile per:

- tracciare operazioni (create/update/delete/share/lock ecc.)
- diagnosticare errori in modo rapido
- avere uno storico leggibile anche senza DBMS esterno

Nota: se la cartella logs/ non esiste, va creata nel progetto prima dell'esecuzione (altrimenti il logger stampa errore su stderr).

9.7 DB Inspectors MapDB

Per facilitare il debug di MapDB sono inclusi due tool avviabili da main():

- com.sweng.notes.debug.NoteDBInspector → ispeziona notes.db
- com.sweng.notes.debug.UserDBInspector → ispeziona users.db

Requisito importante: chiudere Spring Boot prima dell'ispezione

I file MapDB (notes.db e users.db) possono risultare bloccati mentre il backend Spring Boot è in esecuzione.

Prima di lanciare un inspector: fermare il server Spring Boot nel terminale con **CTRL + C**, assicurarsi che l'applicazione sia terminata (nessun processo Java ancora attivo)

Metodo consigliato (semplice): esecuzione da IDE

IntelliJ / VS Code

- aprire la classe NoteDBInspector oppure UserDBInspector
- eseguire il metodo main() (Run)

Output atteso:

stampa "✅ DB trovato: ..."

elenco delle note/utenti in JSON (formattato e leggibile)

Dove cercano i file DB

Gli inspector cercano automaticamente i DB nei percorsi più comuni:

- data/notes.db / data/users.db
(se il working dir è backend/, che è il nostro caso)
- backend/data/notes.db / backend/data/users.db
(se il working dir è la root del repository)

10. Riferimenti e allegati

Questa documentazione (Manuale dello Sviluppatore) è parte di un set di documenti del progetto **Nota Bene**. Per evitare duplicazioni, i contenuti descrittivi (sprint, diagrammi, guida utente) sono mantenuti nei documenti dedicati e qui vengono solo richiamati.

10.1 Diario del Progetto

Documento di riferimento per:

- descrizione degli Sprint 1–5 (attività svolte, risultati, retrospettive);
- organizzazione del lavoro (ruoli, modalità operative, gestione del flusso di sviluppo);
- regole di collaborazione già definite dal team (es. gestione Pull Request).

10.2 Documentazione Sprint 0

(Documentazione Generale – Fase di Inception)

Documento di riferimento per:

- diagrammi e materiale di analisi iniziale (UML, Use Case, modello logico della persistenza/MapDB);
- visione generale del dominio e delle scelte architetturali introduttive.

Nel Manuale dello Sviluppatore i diagrammi non vengono duplicati: per la consultazione completa si rimanda a questo documento.

10.3 Manuale dell'Utente

Documento di riferimento per:

- guida all'utilizzo dell'applicazione dal punto di vista dell'utente finale;
- flussi operativi “user-friendly” (registrazione/login, gestione note, cartelle, condivisione, ecc.);
- indicazioni funzionali senza dettagli di implementazione o codice.

Conclusione

Questo Manuale dello Sviluppatore ha l'obiettivo di fornire un riferimento tecnico chiaro e operativo per installare, avviare e mantenere il progetto **Nota Bene**, permettendo a nuovi sviluppatori di orientarsi rapidamente nel codice e contribuire in modo coerente. Nel documento sono stati descritti l'assetto **client-server** dell'applicazione, la struttura a livelli del backend (**Controller** → **Service** → **Repository**), l'uso dei **DTO** come contratto stabile tra frontend e backend e le principali scelte implementative che governano le funzionalità core (permessi e condivisioni, versionamento/restore, lock concorrente e gestione cartelle).

Per evitare duplicazioni, gli aspetti più narrativi e progettuali (diario degli sprint, analisi iniziale e diagrammi completi) sono mantenuti nei documenti dedicati: **Diario del Progetto**, **Documentazione Sprint 0 (Inception)** e **Manuale dell'Utente**. Questo manuale rimane quindi la base tecnica “di lavoro” per la manutenzione e l'evoluzione del sistema.